

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



SORTING

Oleh:

Muhammad Rifqi Rahman

NIM. 2510817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
TAHUN 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rifqi Rahman
NIM : 2510817310003

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	v
DAFTAR TABEL	vi
KOMPLEKSITAS WAKTU	1
A. Source Code.....	1
B. Output	8
C. Bubble Sort.....	9
D. Selection Sort.....	10
E. Insertion Sort	10
F. Merge Sort	10
G. Quick Sort.....	11
H. Radix Sort.....	11
KARAKTERISTIK ALGORITMA	12
A. Bubble Sort.....	12
B. Selection Sort.....	12
C. Insertion Sort	12
D. Merge Sort	13
E. Quick Sort.....	13
F. Radix Sort.....	13
ANALISIS OPERASI	14
A. Source Code.....	Error! Bookmark not defined.
B. Selection Sort.....	14
C. Insertion Sort	15
D. Merge Sort	15
E. Quick Sort.....	15
F. Radix Sort.....	16
SOAL 4.....	Error! Bookmark not defined.
A. Perbandingan waktu eksekusi antar algoritma	17
B. Algoritma mana yang paling cepat untuk dataset kecil dan besar.....	17

C. Pengaruh distribusi data terhadap performa algoritma.....	17
D. Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.....	18
TAUTAN GIT	20

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Bubble Sort	8
Gambar 2. Screenshot Hasil Selection Sort.....	8
Gambar 3. Screenshot Hasil Insertion Sort	9
Gambar 4. Screenshot Hasil Merge Sort	9
Gambar 5. Screenshot Hasil Quick Sort.....	9
Gambar 6. Screenshot Hasil Radix Sort	9

DAFTAR TABEL

Tabel 1 Source Code sorting_algorithms.cpp.....	1
---	---

KOMPLEKSITAS WAKTU

A. Source Code

Tabel 1 Source Code sorting_algorithms.cpp

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;
6	
7	void bubble_sort(std::vector<int>& data, Metrics& m) {
8	int n = data.size();
9	auto start = Clock::now();
10	
11	for (int i = 0; i < n - 1; ++i) {
12	bool swapped = false;
13	for (int j = 0; j < n - i - 1; ++j) {
14	m.comparisons++;
15	if (data[j] > data[j + 1]) {
16	std::swap(data[j], data[j + 1]);
17	m.swaps++;
18	swapped = true;
19	}
20	}
21	if (!swapped) break;
22	}
23	
24	m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();

```

25 }
26
27 void selection_sort(std::vector<int>& data,
Metrics& m) {
28     int n = data.size();
29     auto start = Clock::now();
30
31     for (int i = 0; i < n - 1; ++i) {
32         int min_idx = i;
33         for (int j = i + 1; j < n; ++j) {
34             m.comparisons++;
35             if (data[j] < data[min_idx]) {
36                 min_idx = j;
37             }
38         }
39         if (min_idx != i) {
40             std::swap(data[i], data[min_idx]);
41             m.swaps++;
42         }
43     }
44
45     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
46 }
47
48 void insertion_sort(std::vector<int>& data,
Metrics& m) {
49     int n = data.size();
50     auto start = Clock::now();
51
52     for (int i = 1; i < n; ++i) {

```



```

53         int key = data[i];
54         int j = i - 1;
55
56         while (j >= 0) {
57             m.comparisons++;
58             if (data[j] > key) {
59                 data[j + 1] = data[j];
60                 m.shifts++;
61                 --j;
62             } else {
63                 break;
64             }
65         }
66
67         if (j + 1 != i) {
68             data[j + 1] = key;
69             m.shifts++;
70         }
71     }
72
73     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
74 }
75
76 void merge(std::vector<int>& data, int left, int
mid, int right, std::vector<int>& temp, Metrics&
m) {
77     int i = left;
78     int j = mid + 1;
79     int k = left;
80

```

```

81     while (i <= mid && j <= right) {
82         m.comparisons++;
83         if (data[i] <= data[j]) {
84             temp[k++] = data[i++];
85         } else {
86             temp[k++] = data[j++];
87         }
88     }
89
90     while (i <= mid) {
91         temp[k++] = data[i++];
92     }
93     while (j <= right) {
94         temp[k++] = data[j++];
95     }
96
97     for (int p = left; p <= right; ++p) {
98         data[p] = temp[p];
99     }
100 }
101
102 void merge_sort_helper(std::vector<int>& data,
103     int left, int right, std::vector<int>& temp,
104     Metrics& m) {
105
106     m.recursive_calls++;
107     if (left >= right) return;
108
109     int mid = left + (right - left) / 2;
110     merge_sort_helper(data, left, mid, temp, m);
111     merge_sort_helper(data, mid + 1, right,
112         temp, m);

```

```

109     merge(data, left, mid, right, temp, m);
110 }
111
112 void      merge_sort(std::vector<int>&      data,
Metrics& m) {
113     auto start = Clock::now();
114
115     if (!data.empty()) {
116         std::vector<int> temp(data.size());
117         merge_sort_helper(data, 0, data.size()
- 1, temp, m);
118     }
119
120     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
121 }
122
123 int partition(std::vector<int>& data, int low,
int high, Metrics& m) {
124     int pivot = data[high];
125     int i = low - 1;
126
127     for (int j = low; j < high; ++j) {
128         m.comparisons++;
129         if (data[j] < pivot) {
130             ++i;
131             std::swap(data[i], data[j]);
132             m.swaps++;
133         }
134     }
135     std::swap(data[i + 1], data[high]);

```

```

136         m.swaps++;
137         return i + 1;
138     }
139
140 void quick_sort_helper(std::vector<int>& data,
    int low, int high, Metrics& m) {
141     m.recursive_calls++;
142     if (low < high) {
143         int pi = partition(data, low, high, m);
144         quick_sort_helper(data, low, pi - 1, m);
145         quick_sort_helper(data, pi + 1, high,
m);
146     }
147 }
148
149 void quick_sort(std::vector<int>& data,
    Metrics& m) {
150     auto start = Clock::now();
151
152     if (!data.empty()) {
153         quick_sort_helper(data, 0, data.size()
- 1, m);
154     }
155
156     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
157 }
158
159 void radix_sort(std::vector<int>& data,
    Metrics& m) {
160     if (data.empty()) return;

```

```

161     auto start = Clock::now();
162
163     int n = data.size();
164
165     int max_val = data[0];
166     for (int i = 1; i < n; ++i) {
167         if (data[i] > max_val) {
168             max_val = data[i];
169         }
170     }
171
172     std::vector<int> output(n);
173
174     for (int exp = 1; max_val / exp > 0; exp *=
175 10) {
176
177         int count[10] = {0};
178
179         for (int i = 0; i < n; ++i) {
180             int digit = (data[i] / exp) % 10;
181             count[digit]++;
182             m.array_accesses++;
183         }
184
185         for (int i = 1; i < 10; ++i) {
186             count[i] += count[i - 1];
187             m.array_accesses += 2;
188         }
189
190         for (int i = n - 1; i >= 0; --i) {
191             int digit = (data[i] / exp) % 10;
192             output[count[digit] - 1] = data[i];

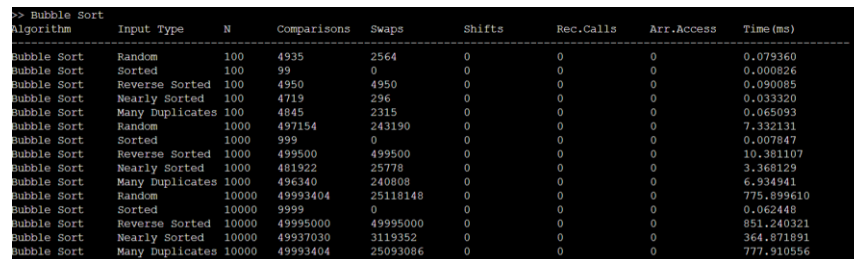
```

```

191         m.array_accesses += 2;
192
193         count[digit]--;
194         m.array_accesses++;
195     }
196
197     for (int i = 0; i < n; ++i) {
198         data[i] = output[i];
199         m.array_accesses++;
200     }
201 }
202
203 m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
204 }

```

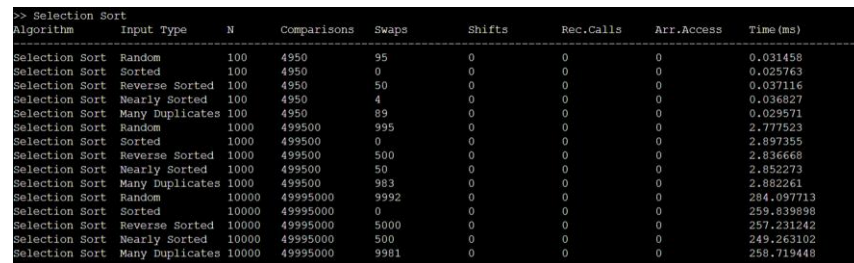
B. Output



The screenshot shows the output of a program testing the performance of the Bubble Sort algorithm across various input types and sizes. The data is presented in a table with columns for Algorithm, Input Type, N, Comparisons, Swaps, Shifts, Rec.Calls, Arr.Access, and Time (ms).

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Bubble Sort	Random	100	4935	2564	0	0	0	0.079360
Bubble Sort	Sorted	100	99	0	0	0	0	0.000826
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.090085
Bubble Sort	Nearly Sorted	100	4719	296	0	0	0	0.033320
Bubble Sort	Many Duplicates	100	4845	2315	0	0	0	0.065093
Bubble Sort	Random	1000	497154	243190	0	0	0	7.332131
Bubble Sort	Sorted	1000	999	0	0	0	0	0.007847
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	10.381107
Bubble Sort	Nearly Sorted	1000	481922	25778	0	0	0	3.368129
Bubble Sort	Many Duplicates	1000	496340	240808	0	0	0	6.934941
Bubble Sort	Random	10000	49993404	25118148	0	0	0	775.899610
Bubble Sort	Sorted	10000	9999	0	0	0	0	0.062448
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	851.240321
Bubble Sort	Nearly Sorted	10000	49937030	3119352	0	0	0	364.871891
Bubble Sort	Many Duplicates	10000	49993404	25093086	0	0	0	777.910556

Gambar 1. Screenshot Hasil Bubble Sort



The screenshot shows the output of a program testing the performance of the Selection Sort algorithm across various input types and sizes. The data is presented in a table with columns for Algorithm, Input Type, N, Comparisons, Swaps, Shifts, Rec.Calls, Arr.Access, and Time (ms).

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Selection Sort	Random	100	4950	95	0	0	0	0.031458
Selection Sort	Sorted	100	4950	0	0	0	0	0.025763
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.037116
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.036827
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.029571
Selection Sort	Random	1000	499500	995	0	0	0	2.777523
Selection Sort	Sorted	1000	499500	0	0	0	0	2.897355
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	2.836668
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	2.852273
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	2.882261
Selection Sort	Random	10000	49995000	9992	0	0	0	284.097713
Selection Sort	Sorted	10000	49995000	0	0	0	0	289.838888
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	257.231242
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	249.263102
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	258.719448

Gambar 2. Screenshot Hasil Selection Sort

```
>> Insertion Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Insertion Sort	Random	100	2656	0	2659	0	0	0.022358
Insertion Sort	Sorted	100	99	0	0	0	0	0.000743
Insertion Sort	Reverse Sorted	100	4950	0	5049	0	0	0.044183
Insertion Sort	Nearly Sorted	100	395	0	375	0	0	0.003583
Insertion Sort	Many Duplicates	100	2412	0	2403	0	0	0.025474
Insertion Sort	Random	1000	244181	0	244184	0	0	2.018444
Insertion Sort	Sorted	1000	999	0	0	0	0	0.006445
Insertion Sort	Reverse Sorted	1000	499500	0	500499	0	0	4.093941
Insertion Sort	Nearly Sorted	1000	26775	0	26766	0	0	0.246853
Insertion Sort	Many Duplicates	1000	241801	0	241794	0	0	2.030349
Insertion Sort	Random	10000	25128135	0	25128140	0	0	209.240317
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.105795
Insertion Sort	Reverse Sorted	10000	49995000	0	50004999	0	0	394.515770
Insertion Sort	Nearly Sorted	10000	3129351	0	3129317	0	0	24.829889
Insertion Sort	Many Duplicates	10000	25103078	0	25103073	0	0	214.161091

Gambar 3. Screenshot Hasil Insertion Sort

```
>> Merge Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Merge Sort	Random	100	535	0	0	199	0	0.018419
Merge Sort	Sorted	100	356	0	0	199	0	0.011511
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.014562
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.017384
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.015512
Merge Sort	Random	1000	8690	0	0	1999	0	0.220705
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.135983
Merge Sort	Reverse Sorted	1000	4832	0	0	1999	0	0.134400
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.170899
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.256779
Merge Sort	Random	10000	120465	0	0	19999	0	3.045771
Merge Sort	Sorted	10000	69008	0	0	19999	0	1.810631
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	1.833760
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	2.134210
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	2.921697

Gambar 4. Screenshot Hasil Merge Sort

```
>> Quick Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Quick Sort	Random	100	678	394	0	129	0	0.012383
Quick Sort	Sorted	100	4950	5049	0	199	0	0.078385
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.049081
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.042113
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.010316
Quick Sort	Random	1000	10537	6125	0	1341	0	0.191758
Quick Sort	Sorted	1000	499500	500499	0	1999	0	7.867804
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	4.774263
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.469555
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.160603
Quick Sort	Random	10000	147606	77996	0	13315	0	2.355100
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	778.423662
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	476.603150
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	7.897834
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	2.164176

Gambar 5. Screenshot Hasil Quick Sort

```
>> Radix Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Radix Sort	Random	100	0	0	0	0	1554	0.009701
Radix Sort	Sorted	100	0	0	0	0	1554	0.009032
Radix Sort	Reverse Sorted	100	0	0	0	0	1554	0.014145
Radix Sort	Nearly Sorted	100	0	0	0	0	1554	0.014270
Radix Sort	Many Duplicates	100	0	0	0	0	1036	0.006910
Radix Sort	Random	1000	0	0	0	0	20072	0.114626
Radix Sort	Sorted	1000	0	0	0	0	20072	0.126449
Radix Sort	Reverse Sorted	1000	0	0	0	0	20072	0.108506
Radix Sort	Nearly Sorted	1000	0	0	0	0	20072	0.108553
Radix Sort	Many Duplicates	1000	0	0	0	0	15054	0.083055
Radix Sort	Random	10000	0	0	0	0	250090	1.383037
Radix Sort	Sorted	10000	0	0	0	0	250090	1.414892
Radix Sort	Reverse Sorted	10000	0	0	0	0	250090	1.528683
Radix Sort	Nearly Sorted	10000	0	0	0	0	250090	1.472556
Radix Sort	Many Duplicates	10000	0	0	0	0	200072	1.143662

Gambar 6. Screenshot Hasil Radix Sort

C. Bubble Sort

Best case Bubble Sort mencapai $O(n)$ ketika data sudah terurut. Dengan optimasi flag, algoritma hanya perlu melakukan satu pass tanpa menemukan satu pun pertukaran, sehingga langsung berhenti. Hal ini dikonfirmasi oleh eksperimen: pada $N=10.000$ dengan input Sorted, hanya diperlukan 9.999 comparisons dan 0 swap dengan waktu 0,062 ms.

Average case berada pada $O(n^2)$. Data acak menyebabkan setiap elemen rata-rata harus dibandingkan dan ditukar sebanyak $n/2$ kali, menghasilkan sekitar $n^2/4$ operasi. Pada

N=10.000 dengan input Random, tercatat 49.993.404 comparisons dan 25.118.148 swaps dengan waktu 775,90 ms.

Worst case $O(n^2)$ terjadi pada data terurut terbalik. Setiap iterasi membutuhkan perbandingan dan pertukaran penuh sebanyak $n(n-1)/2$ operasi. Eksperimen menunjukkan 49.995.000 comparisons dan 49.995.000 swaps (sama besar) dengan waktu 851,24 ms untuk N=10.000 Reverse Sorted.

D. Selection Sort

Best, average, dan worst case Selection Sort semuanya $O(n^2)$. Selection Sort melakukan jumlah comparisons yang sama, yaitu $n(n-1)/2$, karena harus mencari minimum di seluruh sisa array pada setiap iterasi. Eksperimen mengkonfirmasi hal ini: comparisons selalu 49.995.000 untuk semua jenis input pada N=10.000. Yang bervariasi hanya jumlah swaps: input Sorted menghasilkan 0 swap, sementara input Random menghasilkan 9.992 swap.

E. Insertion Sort

Best case $O(n)$ terjadi ketika data sudah terurut. Setiap elemen hanya dibandingkan sekali dengan elemen sebelumnya, menghasilkan $n-1$ comparisons dan 0 shifts. Eksperimen menunjukkan 9.999 comparisons dan waktu 0,11 ms untuk N=10.000 Sorted.

Average case $O(n^2)$ pada data acak, karena rata-rata setiap elemen harus digeser melewati $n/4$ elemen. Tercatat 25.128.135 comparisons dan 25.128.140 shifts (waktu 209,24 ms) untuk N=10.000 Random.

Worst case $O(n^2)$ pada data terurut terbalik, di mana setiap elemen harus digeser melewati seluruh elemen yang sudah terurut sebelumnya. Eksperimen mencatat 49.995.000 comparisons dan 50.004.999 shifts (waktu 394,52 ms) untuk N=10.000 Reverse Sorted. Jumlah shifts sedikit lebih banyak dari comparisons karena ada satu tambahan operasi geser per elemen untuk memindahkan kunci sementara.

F. Merge Sort

Merge Sort memiliki kompleksitas $O(n \log n)$ untuk semua kasus tanpa terkecuali, menjadikannya algoritma yang paling konsisten dan andal. Bahkan pada data sudah terurut, Merge Sort tetap membagi array secara rekursif dan melakukan proses merge, meski dengan jumlah comparisons yang lebih sedikit. Eksperimen menunjukkan comparisons berkisar antara 64.608 (Reverse Sorted) hingga 120.465 (Random) untuk N=10.000, semua jauh di

bawah algoritma $O(n^2)$. Waktu eksekusi sangat konsisten di rentang 1,81–3,05 ms untuk semua jenis input pada $N=10.000$, membuktikan stabilitasnya.

G. Quick Sort

Best case dan average case Quick Sort adalah $O(n \log n)$, terjadi ketika pivot yang dipilih membelah array secara cukup seimbang. Pada input Random $N=10.000$, hanya diperlukan 147.606 comparisons dengan waktu 2,36 ms.

Worst case $O(n^2)$ terjadi ketika pivot selalu menjadi elemen terkecil atau terbesar, menghasilkan partisi yang sama sekali tidak seimbang (satu sisi kosong). Eksperimen membuktikan hal ini secara nyata: input Sorted dan Reverse Sorted pada $N=10.000$ menghasilkan 49.995.000 comparisons masing-masing dengan waktu 778,42 ms dan 476,60 ms. Recursive calls = $19.999 = 2N-1$, membuktikan degenerasi ke kedalaman stack linear, bukan logaritmik.

H. Radix Sort

Radix Sort memiliki kompleksitas $O(nk)$ untuk semua kasus, di mana k adalah jumlah digit dari nilai maksimum dalam array. Kompleksitasnya tidak bergantung pada distribusi data melainkan hanya pada rentang nilai. Eksperimen mengkonfirmasi hal ini: array accesses selalu 250.090 untuk input Random, Sorted, Reverse Sorted, dan Nearly Sorted pada $N=10.000$, nilai yang identik karena semua jenis input tersebut memiliki rentang nilai yang sama. Input Many Duplicates sedikit lebih rendah (200.072) karena nilai-nilai duplikat yang kecil memerlukan lebih sedikit pass digit.

KARAKTERISTIK ALGORITMA

A. Bubble Sort

Bubble Sort bersifat stable karena pertukaran hanya dilakukan ketika $arr[j] > arr[j+1]$ secara ketat (strictly greater than), sehingga elemen-elemen dengan nilai sama tidak pernah ditukar dan urutan relatifnya tetap terjaga. Algoritma ini juga in-place karena hanya membutuhkan $O(1)$ memori tambahan berupa satu variabel sementara untuk proses pertukaran. Mekanisme kerjanya adalah membandingkan pasangan elemen yang berdekatan secara berulang dari kiri ke kanan, lalu menukarnya jika tidak terurut. Proses ini menyebabkan elemen terbesar akan 'menggelembung' (bubble up) ke posisi akhir array pada setiap pass, seperti gelembung yang naik ke permukaan air.

B. Selection Sort

Selection Sort tidak bersifat stable. Saat menemukan elemen minimum dan menukarnya ke posisi depan, operasi ini dapat mengganggu urutan relatif elemen-elemen dengan nilai yang sama. Sebagai contoh, pada array $[3a, 3b, 1]$, swap antara 1 dan 3a menghasilkan $[1, 3b, 3a]$ yang mengubah urutan relatif antara 3a dan 3b. Algoritma ini bersifat in-place karena hanya membutuhkan $O(1)$ memori tambahan. Mekanisme utamanya adalah pada setiap iterasi ke- i , mencari elemen minimum dari subarray $[i..n-1]$ lalu menukarnya ke posisi i . Keunggulan Selection Sort adalah jumlah swap yang sangat minimal, paling banyak hanya $n-1$ kali, sehingga cocok untuk situasi di mana operasi write ke memori sangat mahal.

C. Insertion Sort

Insertion Sort bersifat stable karena elemen hanya digeser jika secara ketat lebih besar dari kunci yang sedang ditempatkan, sehingga elemen dengan nilai sama mempertahankan urutan relatifnya. Algoritma ini in-place dengan $O(1)$ memori tambahan. Mekanismenya adalah membangun array terurut dari kiri ke kanan: setiap elemen baru 'disisipkan' ke posisi yang tepat di antara elemen-elemen yang sudah terurut dengan cara menggeser elemen-elemen yang lebih besar ke kanan. Ini berbeda dari Bubble Sort yang menukar pasangan adjacent; Insertion Sort menggeser secara efisien menggunakan satu operasi shift (bukan tiga assignment layaknya swap), sehingga konstanta tersembunyinya lebih kecil.

D. Merge Sort

Merge Sort bersifat stable karena saat proses merge, ketika dua elemen memiliki nilai sama, elemen dari subarray kiri selalu dipilih terlebih dahulu sehingga urutan relatif terjaga. Algoritma ini tidak in-place karena membutuhkan $O(n)$ memori tambahan untuk array sementara selama proses penggabungan. Mekanismenya menggunakan strategi Divide and Conquer: array dibagi dua secara rekursif hingga ukuran menjadi 1 (basis rekursi), kemudian digabungkan kembali secara terurut. Proses merge membandingkan elemen dari dua subarray yang sudah terurut dan menyusunnya ke array baru, menjamin kompleksitas $O(n \log n)$ yang konsisten.

E. Quick Sort

Quick Sort tidak bersifat stable karena proses partisi dapat memindahkan elemen melewati elemen lain yang bernilai sama, mengubah urutan relatifnya. Secara praktis, algoritma ini in-place karena hanya membutuhkan $O(\log n)$ memori untuk call stack rekursi (bukan memori eksplisit seperti Merge Sort). Mekanismenya adalah memilih sebuah pivot, lalu mempartisi array sehingga semua elemen lebih kecil atau sama dengan pivot berada di kiri dan semua elemen lebih besar berada di kanan. Proses partisi ini diulang secara rekursif untuk kedua bagian. Efisiensi Quick Sort sangat bergantung pada pilihan pivot: pivot yang baik menghasilkan $O(n \log n)$, sedangkan pivot yang buruk (selalu ekstrem) mendegenerasi ke $O(n^2)$.

F. Radix Sort

Radix Sort bersifat stable karena menggunakan Counting Sort yang stabil sebagai subroutine pada setiap pass digit, sehingga urutan relatif elemen dengan digit yang sama selalu dipertahankan. Algoritma ini tidak in-place karena membutuhkan $O(n+b)$ memori tambahan, di mana b adalah basis (biasanya 10 untuk sistem desimal). Mekanismenya adalah non-comparison sort, artinya pengurutan dilakukan tanpa membandingkan nilai secara langsung. Elemen-elemen didistribusikan ke bucket berdasarkan digit tertentu, dimulai dari digit paling tidak signifikan (Least Significant Digit / LSD) hingga digit paling signifikan (Most Significant Digit / MSD). Setiap pass menggunakan Counting Sort untuk menjamin kestabilan distribusi.

ANALISIS OPERASI

analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Bubble Sort

Bubble Sort menggunakan comparisons dan swaps sebagai dua operasi utamanya. Tidak ada shifts, recursive calls, maupun array access yang dihitung secara khusus dalam implementasi ini. Data Sorted merupakan best case: comparisons turun menjadi hanya $n-1$ (berkat optimasi early termination dengan flag) dan swaps = 0, sehingga waktu eksekusi hanya 0,062 ms untuk $N=10.000$. Sebaliknya, data Reverse Sorted adalah worst case absolut di mana comparisons = swaps = $n(n-1)/2 = 49.995.000$, karena setiap elemen harus ditukar sepanjang perjalanannya ke ujung array (waktu 851,24 ms). Data Nearly Sorted menunjukkan pola menarik: comparisons hampir sama dengan worst case (49.937.030) karena tetap harus memeriksa seluruh array, namun swaps jauh lebih sedikit (3.119.352) karena hanya sebagian kecil elemen yang benar-benar salah posisi. Data Many Duplicates menghasilkan comparisons yang tinggi (mirip Random) namun swaps sedikit lebih rendah karena banyak elemen sudah berada di posisi yang mendekati benar.

B. Selection Sort

Selection Sort memiliki karakteristik unik dibanding semua algoritma lainnya: jumlah comparisons selalu konstan untuk ukuran N yang sama, tidak peduli bagaimana distribusi datanya. Yang bervariasi hanyalah jumlah swaps. Comparisons selalu tepat $n(n-1)/2 = 49.995.000$ untuk $N=10.000$ pada semua jenis input tanpa pengecualian. Ini terjadi karena algoritma harus selalu menelusuri seluruh sisa array untuk menemukan elemen minimum, tidak ada jalan pintas. Di sisi lain, swaps sangat bervariasi: input Sorted menghasilkan 0 swap karena minimum selalu sudah berada di tempatnya, Reverse Sorted menghasilkan tepat $n/2 = 5.000$ swap (setiap pasangan posisi pertama-terakhir ditukar), sedangkan Random menghasilkan hingga $\sim n = 9.992$ swap. Meskipun jumlah swaps sangat kecil dibanding

Bubble Sort, waktu eksekusi tetap sekitar 250–284 ms karena didominasi oleh comparisons yang selalu penuh.

C. Insertion Sort

Insertion Sort menggunakan shifts (pergeseran) sebagai pengganti swaps. Satu shift hanya memerlukan satu assignment, sedangkan satu swap memerlukan tiga assignment, sehingga konstanta tersembunyi Insertion Sort lebih kecil dari Bubble Sort. Terdapat pola yang sangat konsisten pada data Insertion Sort: jumlah comparisons $\approx$ jumlah shifts + 1 untuk hampir semua jenis input, menunjukkan bahwa setiap comparison hampir selalu diikuti oleh tepat satu shift. Input Sorted adalah best case terbaik di antara semua algoritma $O(n^2)$: hanya 9.999 comparisons, 0 shifts, dan waktu 0,11 ms untuk $N=10.000$. Input Reverse Sorted adalah worst case dengan 49.995.000 comparisons dan 50.004.999 shifts (jumlah shifts sedikit melebihi comparisons karena ada satu geser tambahan per elemen). Input Nearly Sorted menunjukkan keunggulan paling menonjol Insertion Sort: hanya 3.129.351 comparisons dan waktu 24,83 ms, jauh lebih efisien dari Bubble Sort pada kondisi serupa (364,87 ms). Ini membuktikan bahwa Insertion Sort adalah algoritma terbaik untuk data yang hampir terurut.

D. Merge Sort

Merge Sort menggunakan comparisons dan recursive calls sebagai metrik operasi utama. Jumlah recursive calls selalu $2N-1$, mengikuti sifat binary tree dengan N leaf nodes. Recursive calls selalu persis $2N-1$ untuk semua jenis input ($N=100 \rightarrow 199$, $N=1000 \rightarrow 1999$, $N=10.000 \rightarrow 19.999$), membuktikan bahwa struktur pohon rekursi Merge Sort sama sekali tidak dipengaruhi distribusi data. Yang bervariasi adalah jumlah comparisons: input Reverse Sorted menghasilkan comparisons paling sedikit (64.608 untuk $N=10.000$) sedangkan Random paling banyak (120.465). Ini terjadi karena saat merge dua subarray terurut terbalik, salah satu subarray sering habis lebih cepat sehingga elemen tersisa bisa langsung disalin tanpa perbandingan lebih lanjut. Waktu eksekusi sangat konsisten di rentang 1,81–3,05 ms untuk $N=10.000$ pada semua jenis input, membuktikan kompleksitas $O(n \log n)$ yang tidak bergantung distribusi data.

E. Quick Sort

Quick Sort memiliki variasi operasi terbesar di antara semua algoritma yang diuji karena perilakunya sangat dipengaruhi oleh pilihan pivot dan distribusi data. Input Sorted dan Reverse Sorted merupakan worst case nyata dengan 49.995.000 comparisons masing-masing

dan recursive calls = $19.999 = 2N-1$, membuktikan degenerasi ke kedalaman rekursi linear (bukan logaritmik seperti pada kasus normal). Pada Sorted, pivot pertama selalu elemen terkecil sehingga partisi menghasilkan satu sisi kosong dan satu sisi berisi $N-1$ elemen, mirip Selection Sort. Input Random menunjukkan performa terbaik: hanya 147.606 comparisons dan 13.315 recursive calls (mendekati $\log_2 N \approx 13,3$ level), dengan waktu 2,36 ms. Input Many Duplicates juga menunjukkan performa baik (2,16 ms) karena banyaknya nilai sama menyebabkan pembagian yang lebih seimbang antar partisi. Perbedaan antara Sorted (778 ms) dan Random (2,36 ms) yang mencapai ~330 kali lipat untuk N yang sama merupakan contoh paling dramatis dari pengaruh distribusi data terhadap algoritma.

F. Radix Sort

Radix Sort tidak melakukan comparisons, swaps, shifts, maupun recursive calls. Satu-satunya metrik operasional yang relevan adalah array accesses, mencerminkan mekanisme non-comparison berbasis distribusi digit. Array accesses selalu identik untuk input Random, Sorted, Reverse Sorted, dan Nearly Sorted (1.554 untuk $N=100$, 20.072 untuk $N=1000$, 250.090 untuk $N=10.000$) karena keempat jenis input tersebut memiliki rentang nilai yang sama sehingga jumlah pass digit pun sama. Input Many Duplicates menghasilkan array accesses lebih sedikit (200.072 untuk $N=10.000$) karena nilai-nilai kecil yang mendominasi memerlukan lebih sedikit pass. Formula array accesses secara umum adalah sekitar $(2N + 2 \times \text{basis}) \times k$, di mana k adalah jumlah digit maksimum dan basis = 10. Konsistensi ini membuktikan bahwa Radix Sort benar-benar tidak sensitif terhadap distribusi data, melainkan hanya bergantung pada rentang nilai.

ANALISIS PERFORMA

A. Perbandingan waktu eksekusi antar algoritma

Perbedaan performa antar algoritma sangat besar. Radix Sort secara konsisten menjadi yang tercepat di semua jenis input (1,14–1,53 ms), diikuti Merge Sort (1,81–3,05 ms). Quick Sort sangat kompetitif pada data acak (2,36 ms) namun bisa menjadi yang terburuk pada data terurut (778,42 ms). Di ujung lain, Bubble Sort adalah yang paling lambat secara rata-rata, terutama untuk data tidak terurut (775–851 ms). Selection Sort berada di posisi menengah namun tidak memiliki best case yang berarti (~250–284 ms untuk semua input). Insertion Sort menunjukkan rentang performa paling lebar: dari 0,11 ms (Sorted) hingga 394,52 ms (Reverse Sorted).

B. Algoritma mana yang paling cepat untuk dataset kecil dan besar

Dataset kecil ($N=100$): Radix Sort adalah yang tercepat secara absolut (0,006–0,014 ms) karena overhead-nya kecil dan akses bersifat linear. Insertion Sort pada data Sorted bahkan lebih cepat (0,0007 ms) karena best case-nya hanya $O(n)$. Untuk N sekecil ini, perbedaan antar algoritma $O(n^2)$ tidak signifikan secara praktis, dan overhead rekursi pada Merge Sort maupun Quick Sort relatif terasa lebih besar.

Dataset besar ($N=10.000$): Radix Sort terbaik secara keseluruhan dengan konsistensi tertinggi (1,14–1,53 ms). Merge Sort adalah pilihan terbaik kedua karena tidak ada worst case (1,81–3,05 ms untuk semua input). Quick Sort unggul pada data acak namun harus dihindari untuk data terurut tanpa pemilihan pivot yang lebih cerdas. Insertion Sort sangat unggul untuk data sudah atau hampir terurut, namun buruk untuk yang lainnya. Bubble Sort dan Selection Sort tidak direkomendasikan untuk N besar dalam aplikasi nyata karena kompleksitas $O(n^2)$ yang tidak kompetitif.

C. Pengaruh distribusi data terhadap performa algoritma

Data sudah terurut (Sorted) menguntungkan Bubble Sort dan Insertion Sort secara dramatis karena keduanya memiliki optimasi best case $O(n)$. Bubble Sort hanya membutuhkan 0,062 ms dan Insertion Sort 0,11 ms untuk $N=10.000$, menjadikan keduanya tercepat dalam kategori ini. Sebaliknya, distribusi ini adalah yang paling buruk untuk Quick Sort (778,42

ms) karena pivot selalu menjadi elemen terkecil, menciptakan worst case $O(n^2)$. Radix Sort dan Merge Sort tidak terpengaruh secara signifikan.

Data terurut terbalik (Reverse Sorted) adalah skenario terburuk untuk Bubble Sort (851,24 ms) dan juga buruk untuk Insertion Sort (394,52 ms) serta Quick Sort (476,60 ms). Semua elemen harus bergerak sejauh mungkin dari posisi awalnya. Merge Sort dan Radix Sort tetap efisien di angka 1,83 ms dan 1,53 ms, membuktikan ketahanan mereka terhadap distribusi terburuk sekalipun.

Data hampir terurut (Nearly Sorted) adalah kondisi di mana Insertion Sort bersinar paling terang: hanya 24,83 ms untuk $N=10.000$, jauh lebih baik dari algoritma lain kelas $O(n^2)$. Quick Sort juga berkinerja baik pada kondisi ini (7,90 ms). Namun Bubble Sort masih lambat (364,87 ms) karena meski swaps sedikit, comparisons tetap mendekati $O(n^2)$. Merge Sort dan Radix Sort tetap konsisten seperti biasa.

Data dengan banyak duplikat (Many Duplicates) secara umum tidak memberikan keuntungan signifikan bagi algoritma $O(n^2)$. Quick Sort justru mendapat manfaat besar (2,16 ms) karena banyaknya nilai sama menyebabkan partisi yang lebih seimbang. Radix Sort menjadi yang tercepat (1,14 ms) karena nilai-nilai kecil membutuhkan lebih sedikit pass digit. Merge Sort tetap konsisten (2,92 ms).

D. Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh

Hasil eksperimen secara umum mengkonfirmasi teori kompleksitas dengan akurasi yang baik, meskipun ada beberapa temuan menarik yang layak dicermati.

Konfirmasi $O(n^2)$ pada Bubble Sort: Pada input Random, waktu eksekusi untuk $N=100 \rightarrow 1000 \rightarrow 10.000$ adalah $0,08 \rightarrow 7,33 \rightarrow 775,9$ ms. Rasio kenaikannya adalah $\sim 91\times$ dan $\sim 106\times$ untuk setiap $10\times$ kenaikan N , mendekati teori $O(n^2)$ yang seharusnya menghasilkan rasio $100\times$. Perbedaan kecil ini wajar karena adanya variasi data acak dan faktor hardware seperti cache.

Konfirmasi $O(n \log n)$ pada Merge Sort: Pada input Random, waktu untuk $N=100 \rightarrow 1000 \rightarrow 10.000$ adalah $0,018 \rightarrow 0,22 \rightarrow 3,05$ ms. Rasionya $\sim 12,2\times$ dan $\sim 13,9\times$, sangat

dekat dengan nilai teoritis $10 \times (\log_2 10000 / \log_2 1000) \approx 13,3 \times$. Ini adalah konfirmasi eksperimen yang paling akurat.

Anomali Merge Sort pada Reverse Sorted: Secara mengejutkan, Reverse Sorted menghasilkan comparisons lebih sedikit (64.608) dibandingkan Random (120.465) untuk $N=10.000$. Ini bukan anomali, melainkan konsekuensi logis: saat dua subarray yang masing-masing terurut terbalik digabungkan, salah satu subarray sering kali habis lebih cepat karena semua elemennya lebih besar (atau lebih kecil) dari elemen di subarray lain, sehingga elemen sisa bisa langsung disalin tanpa perbandingan.

Degenerasi Quick Sort pada Sorted: Recursive calls = $19.999 = 2N-1$ untuk input Sorted membuktikan bahwa kedalaman rekursi menjadi linear (N), bukan logaritmik ($\log N$). Ini adalah bukti eksperimental paling jelas dari degenerasi $O(n^2)$ Quick Sort dengan pivot = elemen pertama.

Perbedaan waktu Selection Sort yang terjaga: Meskipun comparisons selalu sama (49.995.000), waktu untuk Sorted (259,84 ms) sedikit berbeda dari Random (284,10 ms). Ini mencerminkan pengaruh cache locality dan branch prediction prosesor: data terurut lebih ramah terhadap prefetcher cache modern sehingga akses memori lebih cepat meskipun jumlah operasi logis identik.

Secara keseluruhan, eksperimen ini mengkonfirmasi bahwa teori kompleksitas adalah panduan yang andal. Untuk penggunaan umum dengan input campuran, Radix Sort atau Merge Sort adalah pilihan terbaik. Quick Sort sangat kompetitif namun memerlukan pemilihan pivot yang lebih cerdas (seperti median-of-three atau random pivot) untuk menghindari worst case pada data terurut. Insertion Sort adalah pilihan optimal untuk data yang sudah atau hampir terurut. Bubble Sort dan Selection Sort sebaiknya hanya digunakan untuk keperluan edukasi atau pada N yang sangat kecil.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-4-sorting-wyattPNG>